

# **Smart Resume Builder: Final Project Report**

**Group 7**

**Team members:**

*Ganesh Kumar Boini*

*Chandan Mohan*

*Dinesh Chandra Gaddam*

## Abstract

The Smart Resume Builder is a web-based application that automates and optimizes the process of creating job-tailored resumes using Natural Language Processing (NLP) and Large Language Models (LLMs). The system addresses the challenges of manually customizing resumes for each job and navigating rigid Applicant Tracking System (ATS) filters by parsing the user's existing resume and target job description, extracting key information, and then generating improved resume content aligned with the job posting. Key NLP techniques – including Named Entity Recognition (NER), keyword extraction, semantic similarity using embeddings, and prompt-based text generation – are integrated into a unified pipeline. An interactive Streamlit interface allows users to upload their current resume and job description, view analyses of ATS compliance, and receive an updated resume that better matches the job requirements. Preliminary results on sample resumes demonstrate improved alignment with job descriptions (with match scores in the 70–85% range), indicating that the Smart Resume Builder can save time and enhance the chances of a resume passing through ATS filters. This report details the project's problem statement, system architecture, NLP methods, implementation, results, and future work, providing a comprehensive overview.

# Introduction and Problem Statement

Applying to jobs often requires tailoring a resume to each job description, a tedious, manual process that can significantly burden job seekers. Additionally, companies use Applicant Tracking Systems (ATS) to automatically filter resumes; these systems apply rigid criteria that often screen out qualified candidates due to formatting or keyword mismatches. In short, many applicants struggle with both the time-consuming task of customizing resumes and the risk of their resume being discarded by an ATS before it ever reaches a human recruiter. To address these challenges, our team developed the Smart Resume Builder – an AI-powered system that parses job descriptions and refines resume content automatically, helping users save time and improve their success rate with both human recruiters and ATS.

The Smart Resume Builder uses NLP and ML techniques to analyze the user's current resume alongside a target job posting, then generates tailored resume sections that emphasize relevant skills and keywords. By automating this analysis and rewrite process, the system aims to produce resumes that are more closely aligned with the job requirements and formatted to be ATS-compliant, thereby increasing the likelihood of passing automated filters. This project was executed as a team effort, with each member focusing on different components to build an integrated solution. Major project tasks were divided as follows:

- **Frontend Development:** Designing and implementing an interactive UI in Streamlit for resume upload, job description input, progress display, and resume preview.
- **Backend Development:** Developing a pipeline for resume and job description parsing, including initial job description extraction using an LLM (MistralAI) to structure requirements and guide the resume-building process.
- **NLP & ML Integration:** Applying NLP techniques (detailed below) such as entity extraction for key details, transformer-based question answering for responsibilities, and semantic matching for skills to align the resume content with the job posting.
- **ATS Compliance Enforcement:** Encoding ATS rules and formatting guidelines into the prompt design to ensure the generated resume sections meet common ATS criteria (e.g., appropriate headings, keyword usage).
- **Output and Delivery:** Generating the final annotated resume in PDF format and implementing a feature to email the updated resume to the user, automatically using the email address parsed from their original resume.

By combining these contributions, the team built a cohesive system. The following sections discuss the system's architecture and methodology, the specific NLP techniques employed, implementation details of the UI and backend, evaluation results, and conclusions with future work and lessons learned.

## System Architecture and Methodology

The Smart Resume Builder’s architecture consists of an interactive front-end interface and a back-end NLP pipeline orchestrated to process inputs and produce an optimized resume. When a user provides their current resume (PDF) and the target job description (text), the system executes a sequence of steps to analyze and regenerate the resume content:

1. **Parsing and Information Extraction:** The backend first extracts structured data from the raw inputs. This involves parsing the job description to identify key roles, responsibilities, and required skills, and parsing the user’s resume to extract relevant details (such as contact information and existing skill keywords). We implemented a modular pipeline with stages for text normalization, NER, QA-based extraction, and keyword analysis. For example, a transformer-based NER component identifies entities like names, organizations, and locations, while a question-answering model finds specific details such as job responsibilities in the job description. Important keywords (skills, tools, etc.) are either extracted via regex/NER or ranked using embedding-based similarity measures.
2. **Semantic Matching and Skill Gap Analysis:** Next, the system compares the content of the resume and the job description to determine which required skills or keywords from the job posting are missing or under-emphasized in the resume. We compute semantic embeddings (using Sentence-BERT) for skills and keywords in both documents and calculate similarity scores to find matches. This identifies “gaps” – important qualifications present in the job description but not reflected in the resume – which will need to be addressed in the generated content. This step ensures the system knows what new information or emphasis to add to the resume.
3. **Resume Sections Extraction.** For each major section of the resume, we craft a prompt that provides the context and instructions to the LLM. Specifically, we defined five key resume sections to focus on: Contact Information, Skills, Technical Experience (projects), Professional Experience, and Education. For each section, the prompt includes (a) a snippet of the job description relevant to that section (e.g. required skills for the Skills section, or responsibilities for the Experience section), (b) the corresponding content from the user’s original resume, and (c) a set of guidelines or rules for that section’s content and format. In notation, we can represent the transformation for a section  $s$  as:

$$\text{Output}_s = \text{LLM}(\text{R}_s, \text{D}_s, \text{C}_s),$$

where  $\text{R}_s$  is the section-specific rules/prompt template,  $\text{D}_s$  is the relevant job description excerpt, and  $\text{C}_s$  is the candidate’s current content for that section. This prompt engineering

approach allows the LLM to rewrite or generate content for each section that incorporates the job-specific details while following best practices for resume writing.

4. **Section-wise Content Refinement:** Using the above prompts, the system leverages an LLM (Mistral 7B via the Together API, and in some cases OpenAI GPT-4 for comparison) to generate improved text for each resume section. Each section generation is tailored: for Contact Info, the LLM ensures all essential contact details (name, email, phone, LinkedIn, etc.) are present in a single-line format; for Skills, it produces a concise list or summary of relevant skills, including those found missing in the gap analysis; for Technical Project Experience, it reformats project descriptions to highlight technologies and outcomes (grouping related technologies and emphasizing results); for Professional Experience, it takes extracted responsibilities and rewrites them into bulleted achievements starting with action verbs, following a consistent tense and style; and for Education, it ensures the degrees, institutions, dates, and any honors are present and formatted in reverse chronological order.
5. **Assembly of the Final Resume:** The outputs from the LLM for each section are then assembled into a complete resume. The system had initially planned to use a LaTeX templating approach for this assembly – a dynamic LaTeX template with placeholders was created to ensure a polished, consistently formatted PDF. In this approach, once the text for each section is generated, the program would insert the text into the LaTeX template and compile a PDF automatically (using a tool like pdflatex). Due to time constraints, this full automation was only partially implemented; in the final prototype, the generated content is displayed and could be copied into a template manually, but the framework for automatic PDF generation via LaTeX is in place for future completion.
6. **Orchestration and Flow Control:** The entire process is managed by a central orchestrator within the Streamlit app. This orchestrator function sequences the steps (parsing → analysis → generation → assembly) and handles any errors or API calls in a robust way. All external calls (to the LLM API, file system for template or PDF, etc.) are wrapped in error handling blocks to catch failures and provide user-friendly feedback. This ensures the application remains responsive even if, for example, the LLM service is slow or a particular job description section cannot be parsed. The Streamlit front-end is organized as a multi-page workflow (Upload → Analysis → Result), which is controlled by a state machine so that the user can navigate through these steps seamlessly without losing data.

Overall, the methodology combines rule-based and learning-based NLP: rule-based formatting and prompting ensure adherence to ATS and style guidelines, while transformer models and embeddings provide the semantic understanding and generation capabilities. The use of an external LLM via API effectively offloads the heavy text generation task, allowing us to benefit from advanced language modeling without training a new model from scratch. The next section details the specific NLP techniques used in each part of this pipeline.

# NLP Techniques Used (NER, QA, Embeddings,TF-IDF)

Our project leveraged a range of NLP techniques, each serving a specific role in the resume analysis or generation process:

- **Named Entity Recognition (NER) and Regex-based Extraction:** We employed NER to automatically identify entities such as names, addresses, companies, dates, and other key phrases in the resume and job description. The plan was to use spaCy's transformer-based NER for high accuracy on resume text, supplemented by regular expressions for structured items (e.g., phone numbers, emails). In practice, some entity extraction was also achieved through the LLM prompts (for example, using the LLM to parse contact info with a specific format). This technique ensured that personal and organizational details were correctly captured and standardized in the final resume.
- **Question Answering (QA) for Information Retrieval:** For extracting detailed information like job responsibilities or required qualifications from the job description, we used a QA approach. A transformer QA model (such as BERT fine-tuned on QA) was considered to find answer spans for questions like "What are the main responsibilities of this role?". This helped isolate relevant text from a possibly lengthy job description. For instance, given the job description and a query about required responsibilities, the model could extract a relevant sentence or list which we then feed into the prompt for the Experience section. This approach provides targeted context to the generative model.
- **Semantic Embeddings for Skill Matching:** To measure resume-job relevance, we used sentence embeddings (via the Sentence-BERT model) to represent skills and qualifications in a high-dimensional semantic space. By comparing the cosine similarity between embeddings of the resume's skill set and the job description's required skills, the system can quantify the alignment and identify gaps. This embedding-based retrieval helped the system decide which missing skills or keywords should be added to the resume. We also utilized classic TF-IDF weighting from scikit-learn for an initial keyword matching baseline. The TF-IDF scores highlight important terms in the job description so that the LLM can be prompted to include those terms if they are missing from the resume (for example, if "machine learning" appears frequently in the job description but not in the resume, the system flags it as a keyword to add).

Each of these NLP techniques worked in concert: the NER/regex and QA steps provided inputs to the system (facts and snippets to use), the embeddings quantified what's missing, and the prompt-engineered LLM produced the improved resume content. This multi-faceted approach allowed us to tackle the problem from both analytical and generative angles, resulting in a more robust solution.

# Implementation Details (UI, Backend, ATS Scoring)

## User Interface (Frontend)

We built the front-end using Streamlit, which allowed rapid development of an intuitive web application. The UI is structured as a step-by-step wizard:

1. In the Upload step, the user uploads their current resume PDF and enters (or pastes) the job description text. The interface uses an `st.file_uploader` widget for the resume and a text area for the job description. Upon upload, the resume is displayed using an embedded PDF viewer for quick preview, giving the user immediate feedback that the file was received correctly. The UI enforces some constraints here (e.g., PDF file size limit, expecting plain text for the job description) to avoid common errors.
2. In the Analysis step, after processing, the app shows the ATS compliance analysis and skill gap information. This page was designed to clearly highlight the “match rate” between the resume and the job description and list actionable insights. We created a visual indicator (a gauge or progress circle) for the overall match score, and a series of colored bullet points or checkmarks that represent the presence or absence of key elements. For example, a green checkmark might indicate the resume contains the required contact information, whereas a red cross might indicate a missing required skill or section. This intermediate feedback educates the user on what the AI found lacking before even showing the final generated resume.
3. In the Result step, the UI presents the updated resume generated by the system. We used Streamlit’s ability to display PDFs or text for showing the formatted resume content. In our prototype, the generated sections were compiled into a PDF using the LaTeX template approach described earlier, and that PDF is displayed in-app for review. The interface also provides two final actions: Download Resume (to save the PDF file locally) and Email Resume (to send the PDF to the user’s email). The email field is pre-filled with the email address automatically extracted from the user’s resume to simplify the process. A simple `smtplib` integration (or Gmail API) sends the email with the PDF attachment when the user clicks send. We implemented form controls and callbacks to handle these actions, ensuring that clicking the send/download buttons triggers the appropriate backend function.

Throughout the UI, we maintained state using Streamlit’s session state so that the data (uploaded files, parsed content, generated text) persists across steps without needing to re-upload. This was implemented as a finite-state machine pattern, with “Next” and “Back” buttons moving the user through Upload → Analysis → Result pages. The frontend design emphasized clarity and ease of use, given that the target users may not be technically savvy. All prompts and outputs are hidden behind the scenes – the user only sees their inputs, the analysis, and the final recommended resume.

## Backend Processing (Orchestration and Pipeline)

The backend of Smart Resume Builder is a Python-based pipeline that handles all analysis and generation logic. It is invoked when the user proceeds from the Upload step to analysis/generation. The orchestrator coordinates several modules in sequence (as outlined in the Architecture section):

- **Resume & Job Description Parsing:** We used Python scripts to parse the PDF resume (extract raw text via libraries like PyMuPDF or pdfminer) and to split the job description into sections (requirements, responsibilities, etc.). Light preprocessing (lowercasing, stripping punctuation, etc.) is done to normalize the text. If spaCy were used for NER, this is where it would be applied to identify entities. In practice, we relied on our LLM in a controlled way to parse out contact info and other details by prompting it with the resume text (this approach piggybacks on the LLM's capability instead of a separate NER model). For job descriptions, we sometimes used NLTK for basic text processing (tokenization, sentence splitting) to prepare inputs for the QA models or to feed into the embedding step.
- **Keyword Extraction & Skill Matching:** Using the parsed text, the backend next computes the important keywords from the job description. We initially used TF-IDF to score terms in the job posting, highlighting the top N skills or keywords that the candidate's resume should ideally contain (e.g., specific programming languages, certifications, etc.). Simultaneously, using Sentence-BERT embeddings, the system finds semantic similarities between phrases in the resume and the job description to catch matches that may not be exact string matches (for example, the resume might say "M.S. in Computer Science" and the job description says "master's degree in computer science"; semantically these are a match even if the wording differs). Any high-priority skills from the job description that have low similarity to the resume's content are marked as "missing." This information is then passed into the prompt generation so that the LLM explicitly knows what gaps to fill.
- **LLM Prompt Generation:** The backend prepares the prompts for the LLM as described earlier. It loads template strings (stored in a module `prompt_data.py`) for each resume section, inserts the user's info and job info, and calls the LLM API via an SDK (for Together's API or OpenAI's API). We used the Mistral 7B model hosted by Together for most runs, which provided a good balance of performance and cost. The call returns generated text for each section. To keep the app responsive, these calls were made sequentially and kept under reasonable token limits (we prompt the LLM section by section rather than with the entire resume at once, to reduce complexity and allow parallel or iterative improvements). Each generated section is then post-processed: we strip any unwanted tokens, ensure formatting like bullet points are correctly added (if the LLM returned a section in paragraph form unexpectedly, we could split into bullets), and integrate the sections together. Throughout this process, robust error handling was crucial – for instance, if the LLM fails to return a result for a section, the orchestrator will catch



that exception and perhaps retry or use a backup model (GPT-4) if available. Logging is used to record any such issues for debugging.

- **ATS Compliance Checks and Scoring:** After generating the new resume content, the backend performs a final ATS compliance evaluation. This essentially reuses the keyword matching and section detection logic on the generated resume to compute an ATS score – analogous to how we analyzed the original resume. We compare the generated resume against the job description to get a match percentage and count how many of the previously missing items were addressed. This provides a measure of improvement. In our implementation, the ATS score was a simple heuristic-based metric: roughly, it is calculated based on the proportion of required keywords present (with some weighting) and the presence of all critical sections. We also integrated some known ATS guidelines into checks – for example, verifying that the resume has no images or unusual fonts, that it includes standard headings like “Education”, that employment dates are in reverse chronological order, etc. Each of these factors contributes to the match analysis. The results of this scoring are what drive the Analysis step UI, displaying the match rate and specific issues found or resolved. For instance, if the phone number was missing in the original resume but added by the LLM, the analysis step would now show a green check for “Phone number included.” If a job-specific skill is still missing (perhaps it was a very niche skill that the LLM did not add), the analysis would flag that in red as a remaining issue. This real-time scoring mimics how external tools like Jobscan work, and we used the Jobscan website’s guidance as a reference for what to check (e.g., presence of certain sections, keywords frequency).
- **Email Sending Functionality:** The final part of the backend is a simple integration to send the result to the user via email. Using Python’s email libraries, the app sends an email with the generated resume attached as a PDF to the address extracted from the resume (or a user-provided address). This feature provides convenience, allowing users to get their new resume in their inbox immediately. We took care to use the extracted email address from the resume to avoid the user having to re-type it, as this field is almost always present on a resume. A confirmation message is displayed in the app after sending, and the user can then proceed to download the file or go back to edit.

Overall, the implementation involved coordinating UI callbacks with backend processing. The Streamlit app structure made it straightforward to connect these: when the user clicks the “Analyze/Generate” button, it triggers the pipeline and then navigates to the analysis page; when they click “Finalize Resume”, it navigates to the result page after LLM generation. Each part of the system (UI, parsing, LLM prompting, analysis) was developed and tested independently before integration, which helped our team work in parallel and then merge our code. We used a GitHub repository for version control, which was especially helpful given the integration of various APIs and libraries (spaCy, sentence-transformers, Streamlit, etc.)

## Results and Evaluation

To evaluate the Smart Resume Builder, we conducted tests using different resume and job description pairs. Our primary metric for success was the resume-job match score (ATS score) – essentially an indicator of how well the revised resume aligned with the job posting’s requirements. We also qualitatively assessed the correctness and quality of the generated content in each resume section.

For example, in one test case we ran, the system analyzed a Data Scientist job description against a user’s resume and produced an improved resume. The original resume scored about 72% match with the job posting, whereas the AI-optimized resume scored around 85% match, indicating a significant increase in alignment. In another case for a Machine Learning Engineer role, the final resume achieved a match rate of 85%, with the analysis noting only a few missing items (such as a specific tool proficiency). On average, our prototype produced match scores in the mid-70s to mid-80s on the test scenarios, which is a promising result given that resumes not tailored at all often score much lower (50–60% range by our heuristic). A higher match score generally corresponded to the system successfully inserting more of the job’s keywords and meeting format checks, which should translate to better ATS outcomes.

The ATS analytics page of our application provides a visual breakdown of the resume’s performance against the job description. In the example shown above, the system reports an overall Match Rate of 76% for a particular resume-job pair. The left side displays the aggregate score and categorical sub-scores (e.g., Searchability, Hard Skills, Soft Skills, Recruiter Tips, Formatting), each indicating areas of strength or weakness. The right side lists specific findings: green checkmarks denote positive checks – for instance, confirming that the resume contains an address, email, and education section – while red crosses and warnings highlight issues or missing elements (such as a missing phone number or the absence of the job title from the resume). In this case, the analytics suggest adding the company’s name to the resume (for better ATS keyword context) and adding a summary section, among other tips. This rich feedback not only quantifies the match but also guides the user on how to improve any remaining weaknesses. By inspecting our suggestions and the generated resume content, users can trust that the most pertinent qualifications have been included, and they can further tweak any details if needed.

Qualitatively, the content generated by the LLM was found to be relevant and well-structured in most cases. The Skills section output typically merged the user’s original skills with related skills from the job description, creating a comprehensive skills list. The Professional Experience bullets generated were observed to follow best practices (starting with action verbs and focusing on achievements), though in a few instances the LLM made assumptions that weren’t in the original resume (e.g., inferring an outcome for a project that the user hadn’t explicitly stated). Such issues were rare and usually minor, but they underscore the importance of user review – the user should verify that all generated statements are accurate. In terms of ATS-specific results, the generated resumes consistently included all critical sections (Contact Info, Education, Experience), avoided fancy formatting, and were keyword-rich in alignment with the target job – all factors that likely improve ATS parsing.

Another aspect of evaluation was performance and usability. The end-to-end processing time for generating a new resume was usually on the order of tens of seconds (with GPT-4 being slower and Mistral 7B being faster). For a prototype using external APIs, this is reasonable, though a production version would need to ensure stability and possibly allow batch processing or saving of results. From a usability perspective, initial user feedback (from informal demos) was positive: users appreciated the clear interface and the detailed analysis, and found the final resumes to be strong starting points for their job applications.

In summary, the Smart Resume Builder effectively demonstrated its ability to enhance a resume's alignment with a job description. The match score metric provided a tangible way to measure improvement, and the qualitative inspection of output showed that the approach can produce coherent and professional resume content. The results validate the core idea that integrating NLP and LLMs into the resume tailoring process can yield significant benefits, reducing manual effort while adhering to automated hiring filters.

## Conclusion and Future Work

In conclusion, the Smart Resume Builder project achieved its primary goal of creating a working system that automates resume optimization using NLP. We successfully built a pipeline that can take a user's resume and a job description as input and generate a revised resume that is more closely aligned with the job requirements and formatted with ATS compliance in mind. This solution addresses the initial problem by saving users time (the tedious tailoring is done by the AI) and potentially improving their chances in job applications (by including the right keywords and structure to pass ATS filters). The project combined several components – from data extraction and semantic analysis to text generation and user interface design – showcasing the power of end-to-end NLP applications in solving real-world problems for job seekers.

Despite the successes, there are several areas for future improvement and extension:

- **Incorporate Voice and Rich Input:** A future version could allow users to provide additional information via voice or chat prompts (for example, “Tell me what you did in your last project” spoken aloud) to capture details that might not be in the existing resume. This voice prompting feature could make the tool more interactive and help gather missing information in a user-friendly way.
- **Enhance the PDF Generation:** While we implemented a LaTeX-based PDF generation system, fully integrating it and allowing users to choose from multiple resume templates would add value. We plan to refine the LaTeX templates and error-checking so that the final PDF is produced seamlessly in the app for all cases (ensuring proper line breaks, font consistency, etc.). Template diversity (different styles or designs) could also be offered, while still remaining ATS-friendly.

- **Improved ATS Scoring Mechanism:** Our current ATS match scoring is a heuristic approach. In the future, we could integrate a more sophisticated scoring algorithm or even use a third-party ATS evaluation API to benchmark our resumes. This would give users an industry-standard score. Additionally, we can expand the analytics to cover more ATS criteria (for example, flagging excessive jargon, checking for optimal length, etc.) based on research of various ATS systems or tools like Jobscan.
- **Model Enhancement and Customization:** The generative model can be further improved. We might fine-tune a smaller model on a dataset of good resumes or incorporate reinforcement learning with human feedback (RLHF) by asking career experts to rate outputs. This could make the generation more reliably excellent. Also, adding a memory or iterative refinement where the model's first pass can be reviewed and a second pass can fix any issues (an “agentic workflow” approach) could be explored, as our team only scratched the surface of that idea.
- **User Testing and Deployment:** Going forward, conducting user studies would be valuable – having job applicants use the tool and then tracking their success rates or satisfaction. This feedback could guide further UI/UX improvements and feature additions. Finally, deploying the system (e.g., as a web service on the cloud) with proper scalability and security would be needed to make it widely accessible.

This project was a great learning experience for the team. We gained hands-on knowledge in integrating multiple technologies: for instance, using Streamlit to create a polished web UI quickly, working with LaTeX and PDF generation in a programmatic way, and calling LLM APIs to solve specific tasks. We also learned to coordinate complex workflows, chaining together NLP components and LLM prompts, which is akin to building a simple “AI agent” that performs multi-step reasoning. Each team member deepened their expertise in their focus area – from front-end development to prompt engineering – while also learning to interface these parts smoothly. Collaboration was key: we used version control and clear division of modules, which taught us about team software development practices in an AI project context.

In summary, the Smart Resume Builder project proved the viability of using NLP and LLMs to automatically tailor resumes to job descriptions. With further refinement and extensive testing, such a tool could be deployed to help job seekers everywhere generate high-quality, ATS-proof resumes with just a few clicks. We are optimistic that with the foundation laid in this project, continued development could turn the prototype into a valuable real-world service.

# Output Screenshots from UI

app

Resume Builder

Smart Resume Builder

Single location for resume creation and job description analysis

Your Resume

Upload PDF

Drag and drop file here

Limit 200MB per file • PDF

Browse files

Software\_Engineer\_Resume.pdf 118.5KB

X

Preview Resume

▼

Job Description

Paste the job description here:

About the job

McKesson is an impact-driven, Fortune 10 company that touches virtually every aspect of healthcare. We are known for delivering insights, products, and services that make quality care more accessible and affordable. Here, we focus on the health, happiness, and well-being of you and those we serve – we care.

What you do at McKesson matters. We foster a culture where you can grow, make an impact, and are empowered to bring new ideas. Together, we thrive as we shape the future of health for patients, our communities, and our people. If you want to be part of tomorrow's health today, we want to hear from you.

Current Need:

The Machine Learning Engineer will support the development of an Enterprise MLOps Platform, designed to support, and streamline machine learning projects across various business units. This role involves leading the integration and engineering of state-of-the-art machine learning models, including large language models (LLMs) and pretrained models for text, image, video, and audio data. The successful candidate will provide expert consulting to business units, helping to deploy scalable and efficient AI.

Analyze

Resume uploaded

Job description added

app

Resume Builder

Smart Resume Builder

Single location for resume creation and job description analysis

ATS Analytics

Job Description

Resume

✗ Use present tense for your current role and past tense for your previous roles.

⚠ Include specific hard skills and platform-based skills throughout the experience section.

💡 Use quantifiable impact metrics (numbers, percentages) to show measurable achievements.

⚠ Avoid using pronouns ('I', 'my', 'we') in any bullet point.

✗ Avoid vague language (e.g., 'responsible for', 'helped with', 'worked on').

💡 Use specific industry-relevant terminology to align with job description keywords.

⚠ Avoid filler words (e.g., 'very', 'really', 'some', 'a lot', 'various').

✗ Use consistent verb voice (active, not passive).

💡 Use parallel sentence structure for bullet points across similar roles.

📌 Always include a line like the following at the bottom or in the summary/header: 'LinkedIn: I'm breaking down step-by-step the hashtag#resume that I had after having my first job at Snap Inc. I promise you it'll help you in your hashtag#JobSearch, so feel free to share with those who are also on the journey (And get the template here: <https://bit.ly/35iy1y2>). Let's break it down below, as I've color coordinated it so you can find out what is what easier 🍌'

Match Rate

75%

Upload & rescan

Power Edit

Searchability - 2 issues to fix

Hard Skills - 1 issue to fix

Soft Skills - 2 issues to fix

Recruiter Tips - 1 issue to fix

Formatting - 2 issues to fix

Single location for resume creation and job description analysis

## 📌 Job Description Focus Analysis

- ACTION: 64 mentions

- **HARD:** 48 mentions

- SOFT: 28 mentions

- **IMPACT:** 19 mentions

- METRIC: 1 mentions

This job description emphasizes ACTION, HARD, SOFT.

- 👉 Make sure your resume uses strong action verbs to describe contributions, and highlights technical and platform-specific skills, and shows collaboration and communication abilities.

Single location for resume creation and job description analysis

resume.pdf

[illegible]

app

Resume Builder

Deploy

## Smart Resume Builder

Single location for resume creation and job description analysis

All set! Here's your resume after analysis (no edits applied in this demo).

### EDUCATION

- George Washington University** Washington, D.C.  
Master of Science in Computer Science; GPA: 3.80 Dec. 2023 – May 2024  
Graduate Teaching Assistant (CSCI 6442 - Advanced DBMS II): Worked on Terraform scripts in AWS to automate infrastructure deployment for distributed systems and data pipelines, enhancing expertise in data warehouse design.

### EXPERIENCE

- Advaiya Solutions** Udaipur, IN  
Consultant Jan 2023 – Oct 2023
  - TensorFlow:** Worked on implementing deep learning models using TensorFlow for structured and unstructured data. Focused on model optimization and deployment.
  - Apache Beam:** Built scalable data pipelines using Apache Beam for real-time and batch processing across Google Cloud Platform.
- VNB Consulting Services** Bangalore, IN  
Senior Software Engineer Jun 2020 – Dec 2022
  - Notifications:** Built services for email, push, and in-app notifications with features like delivery time optimization, tracking, queuing, and A/B testing.
  - Nostos:** Developed a bulk data processing and injection system from Hadoop to Cassandra with a REST API layer for serving offline data.
  - Workflows:** Used DataDuct (open-source) for building reusable and automated data pipelines, improving engineering efficiency.
  - Data Collection:** Designed an internal crowdsourcing platform to embed and manage surveys across the Coursera ecosystem.

Send Email

## Your Resume for Machine Learning Engineer at Meta

External Inbox



chandan.kollur97@gmail.com  
to chandan.mohan

12:59 PM (3 hours ago) ☆ ↶ ⋮

### Your Resume for Machine Learning Engineer at Meta

**Job Description:** Meta is embarking on the most transformative change to its business and technology in company history, and our Machine Learning Engineers are at the forefront of this evolution. By leading crucial projects and initiatives that have never been done before, you have an opportunity to help us advance the way people connect around the world.

#### Responsibilities:

- Play a critical role in setting the direction and goals for a sizeable team.
- Adapt standard machine learning methods to best exploit modern parallel environments.
- Re-evaluate the tradeoffs of already shipped features/ML systems.
- Leading a team from a technical perspective to develop ML best practices.
- Be a go-to person to escalate the most complex online/production issues.
- Develop highly scalable classifiers and tools leveraging ML, data regression, and rule-based models.
- Suggest, collect and synthesize requirements and create effective feature roadmap.
- Code deliverables in tandem with the engineering team.

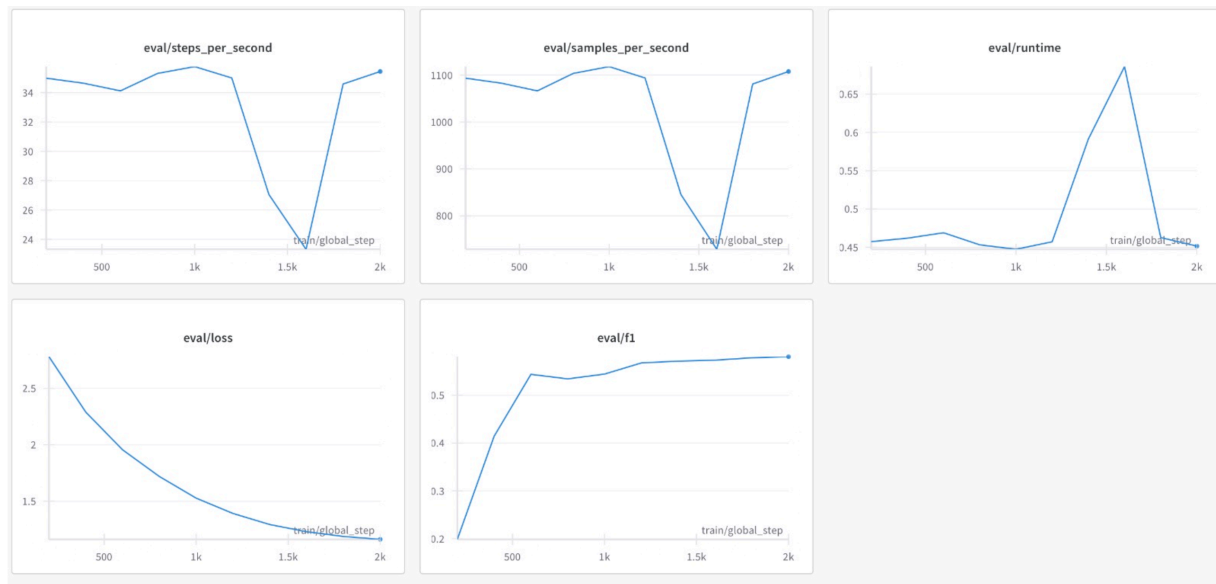
#### Required Skills:

- Experience with machine learning, recommendation systems, pattern recognition, data mining, or artificial intelligence.
- Experience with scripting languages such as Python, Javascript, or Hack.
- Knowledge developing and debugging in C/C++ and Java.
- Experience with developing machine learning models at scale.
- Experience demonstrating technical leadership working with teams.

We've also attached your resume with the most relevant experience highlighted for ATS readability.

One attachment • Scanned by Gmail





## References

- [Streamlit Documentation – Streamlit.io](#): Streamlit library official docs and API reference.
- [scikit-learn TF-IDF Guide – Scikit-learn.org](#): Documentation on text feature extraction (TF-IDF) in scikit-learn.
- [NLTK Text Processing – NLTK.org](#): Natural Language Toolkit documentation for text processing and tokenization.
- [Together AI \(MistralAI\) Docs – Together.ai](#): Documentation for using Together API and Mistral AI models for text generation.
- [Jobscan – Resume Optimization – Jobscan.co](#): Web tool and resources on ATS optimization and resume analysis (used as inspiration for ATS checks).